

Deux architectures, deux niveaux

Le socle est commun. Ce qui distingue les équipes, c'est où vit le **cerveau de l'agent** — orchestré dans n8n, ou codé dans le front. Un seul choix par équipe.

— SOCLE COMMUN — QUELLE QUE SOIT L'OPTION

> Front **Next.js** • déploiement Vercel

> Données **Airtable** / **Supabase**

> Tool **calculer_devis()** déterministe

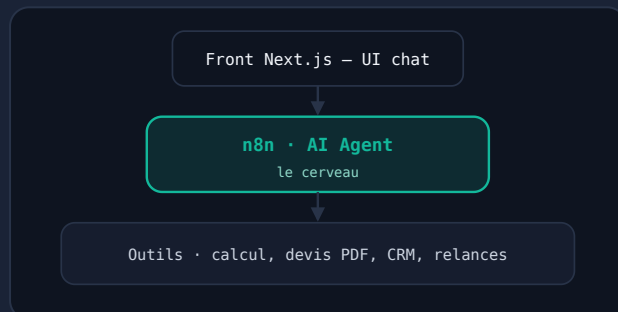
> Devis PDF auto-généré

> Garde-fou : le LLM décide, le code calcule

BON NIVEAU

Option A — n8n au cœur

L'orchestration vit dans n8n



Ce qui change : le nœud AI Agent de n8n mène la conversation et appelle tous les outils. Le front Next.js n'est qu'une **UI de chat** branchée sur un webhook n8n.

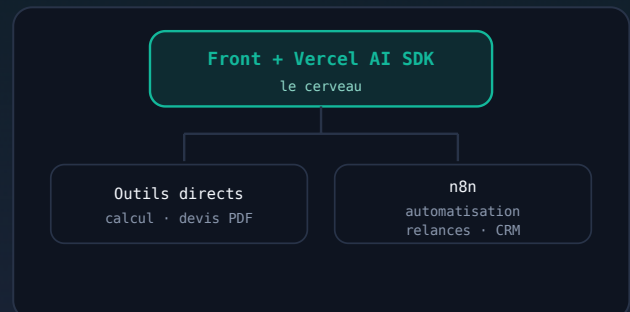
- + Orchestration **visuelle**, agent centralisé au même endroit
- + Courbe d'apprentissage douce, résultat rapide
- Streaming OK via le widget n8n ; UI 100 % sur-mesure = intégration plus lourde
- Part de **code / dev** plus mince

Pour qui : une équipe qui vise un résultat solide et maîtrisé, sans surcharge de code.

EXCELLENT NIVEAU

Option B — Vercel AI SDK au cœur

L'agent vit dans le code



Ce qui change : l'agent (streaming + tool calling, dont **calculer_devis()**) est codé dans Next.js via le SDK. n8n redevient le **dos automatisé** : relances planifiées, écritures CRM, notifications.

- + Vrai **vibe coding** — la compétence dev au premier plan
- + Chat fluide (streaming natif) • garde-fou **littéral dans le code**
- Plus exigeant : état conversationnel à gérer
- S'éloigne du tout-n8n de l'énoncé d'origine

Pour qui : une équipe à l'aise en code, qui vise l'excellence et une expérience produit aboutie.

À RETENIR

Dans les deux cas, **calculer_devis()** reste un **tool déterministe exécuté par du code**. L'option B rend ce principe littéral : la fonction **execute** du tool, c'est ton code — le modèle décide de l'appeler, le code calcule. **Choisissez une seule option** et tenez-la : dupliquer la logique d'agent dans n8n ET dans le SDK est le piège à éviter.

Quel outil, où, dans chaque option

Le cerveau change (n8n ou AI SDK), mais chaque brique reste assurée. Voici l'outil concret derrière chaque rôle.

BRIQUE	OPTION A – N8N AU CŒUR	OPTION B – AI SDK AU CŒUR
Calcul du prix	Nœud Code n8n (JS) = <code>calculer_devis()</code>	Fonction TS exposée comme tool (<code>execute()</code>) du AI SDK
Devis PDF	Nœud n8n HTML→PDF (ou service de templating)	Lib Node dans Next.js (Puppeteer / react-pdf)
CRM (données)	Base Airtable · écriture via le nœud Airtable	Supabase (ou Airtable) · via client serveur Next.js
Dashboard pilotage	Airtable Interface (no-code)	Page React / Next sur-mesure (lit Supabase)
Relances	Schedule Trigger n8n + envoi email (SMTP / Gmail)	n8n en back-office (Schedule + email) ; le front délègue

— TROIS PRÉCISIONS

- > **Données = Airtable OU Supabase.** Airtable (no-code) va de pair avec son Interface comme dashboard ; l'Interface Airtable ne lit que des données Airtable. Supabase (relationnel) impose un dashboard custom React/Next — qui, lui, sait lire l'un ou l'autre.
- ! **Relances : email par défaut** (gratuit, trivial à brancher). WhatsApp possible « pour aller plus loin », mais l'API officielle a un coût et une vérification Meta — hors périmètre d'une semaine.
- > **En option B :** le front Next.js gère calcul + PDF + CRM ; n8n se concentre sur les **relances planifiées**. C'est cohérent avec « l'agent vit dans le code » — à inverser si vous préférez réutiliser n8n comme exécutateur.